UT3. PROGRAMACIÓN MULTIHILO

ACTIVIDAD 7

Prioridades de los hilos

Título: Carrera de hilos: prioridades.

Descripción:

En esta actividad, simularás una carrera entre varios corredores utilizando hilos. Cada corredor será representado por un hilo y tendrá una prioridad asignada, que simula su "nivel de energía". La meta es entender cómo las prioridades afectan la ejecución de los hilos en Java.

Objetivos:

- Comprender el concepto de prioridades en los hilos y su efecto en la ejecución.
- Implementar y ajustar prioridades de hilos en un programa Java.
- Observar cómo el planificador de hilos utiliza las prioridades para decidir el orden de ejecución.

Instrucciones:

Escenario:

Imagina que estás organizando una carrera entre 5 corredores, donde cada corredor tiene un nivel de energía (prioridad) diferente. Tu tarea será programar esta carrera en Java utilizando hilos con prioridades.

1. **Crea una clase Corredor que implemente Runnable:**
   - Cada hilo debe imprimir mensajes en la consola, indicando su progreso en la carrera, por ejemplo:

     "*Corredor 1 ha avanzado al punto 2.*".

2. **En el método main:**
   - Crea 5 instancias de Thread, asignando a cada hilo una prioridad diferente usando setPriority().

     Asigna las siguientes prioridades a los corredores:

     Corredor 1: Thread.MIN_PRIORITY
     Corredor 2: Thread.NORM_PRIORITY
     Corredor 3: Thread.MAX_PRIORITY
     Corredor 4: Thread.NORM_PRIORITY
     Corredor 5: Thread.MIN_PRIORITY

- Los corredores con mayor prioridad deberían tener más probabilidades de ganar la carrera.

3. **Simula la carrera:**
   - Haz que cada hilo avance en etapas (simulando tiempo de carrera con Thread.sleep()), hasta que todos alcancen la meta. Incluye 10 etapas intermedias.
4. **Imprime el orden de finalización:**
   - Usa una estructura compartida, como una lista sincronizada, para registrar el orden en que los corredores cruzan la línea de meta.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Implementa parcialmente hilos, pero no utiliza prioridades ni sincronización
5	Implementa correctamente los hilos con prioridades y registra su orden de llegada, pero no incluye capturas de pantalla ni responde a las preguntas.
7	Integra el uso de prioridades de hilos y asegura un correcto manejo del orden de llegada (sincronizada), pero no responde a las preguntas.
9	Completa la tarea correctamente en todos sus ámbitos.
10	Agrega funcionalidades extra, como personalización dinámica de prioridades o una interfaz interactiva para iniciar la carrera.

Entrega:

i. Incluye aquí el código fuente:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class CarreraHilos {
```

```
    // Lista sincronizada para registrar de forma segura el orden de
llegada de los corredores
    private static final List<String> ordenLlegada =
Collections.synchronizedList(new ArrayList<>());
```

```
    static class Corredor implements Runnable {
        private String nombre;
```

```
        public Corredor(String nombre) {
```

```
            this.nombre = nombre;
        }

        @Override
        public void run() {
            try {
                for (int etapa = 1; etapa <= 10; etapa++) {
                    Thread.sleep((long) (Math.random() * 100 + 50));
                    System.out.println(nombre + " ha avanzado al punto "
+ etapa);
                }

                ordenLlegada.add(nombre);
                System.out.println(nombre + " ha cruzado la meta");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.out.println(nombre + " se a interrumpido");
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {

        System.out.println("Autor: Juan Gabriel Galarza Claros");

        // Crear 5 hilos
        Thread corredor1 = new Thread(new Corredor("Corredor 1"));
        Thread corredor2 = new Thread(new Corredor("Corredor 2"));
        Thread corredor3 = new Thread(new Corredor("Corredor 3"));
        Thread corredor4 = new Thread(new Corredor("Corredor 4"));
        Thread corredor5 = new Thread(new Corredor("Corredor 5"));

        // Poner prioridades
        corredor1.setPriority(Thread.MIN_PRIORITY);
        corredor2.setPriority(Thread.NORM_PRIORITY);
        corredor3.setPriority(Thread.MAX_PRIORITY);
        corredor4.setPriority(Thread.NORM_PRIORITY);
        corredor5.setPriority(Thread.MIN_PRIORITY);

        System.out.println("Comienza la carrera de hilos\n");

        corredor1.start();
        corredor2.start();
        corredor3.start();
        corredor4.start();
        corredor5.start();

        corredor1.join();
        corredor2.join();
        corredor3.join();
```

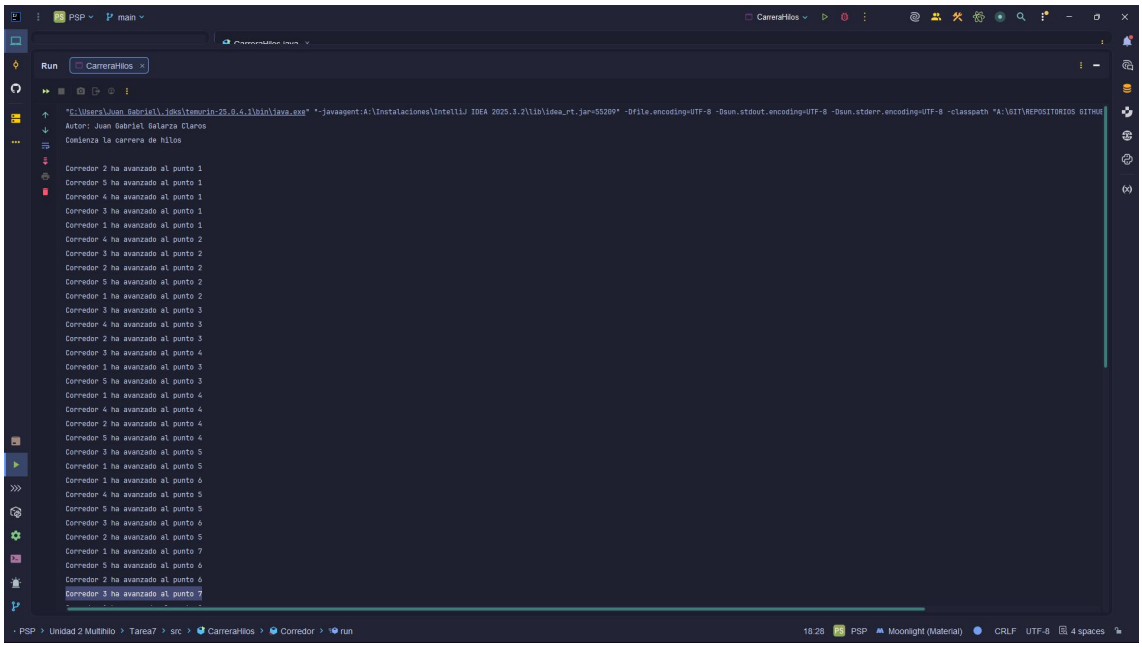
```
        corredor4.join();
        corredor5.join();
```

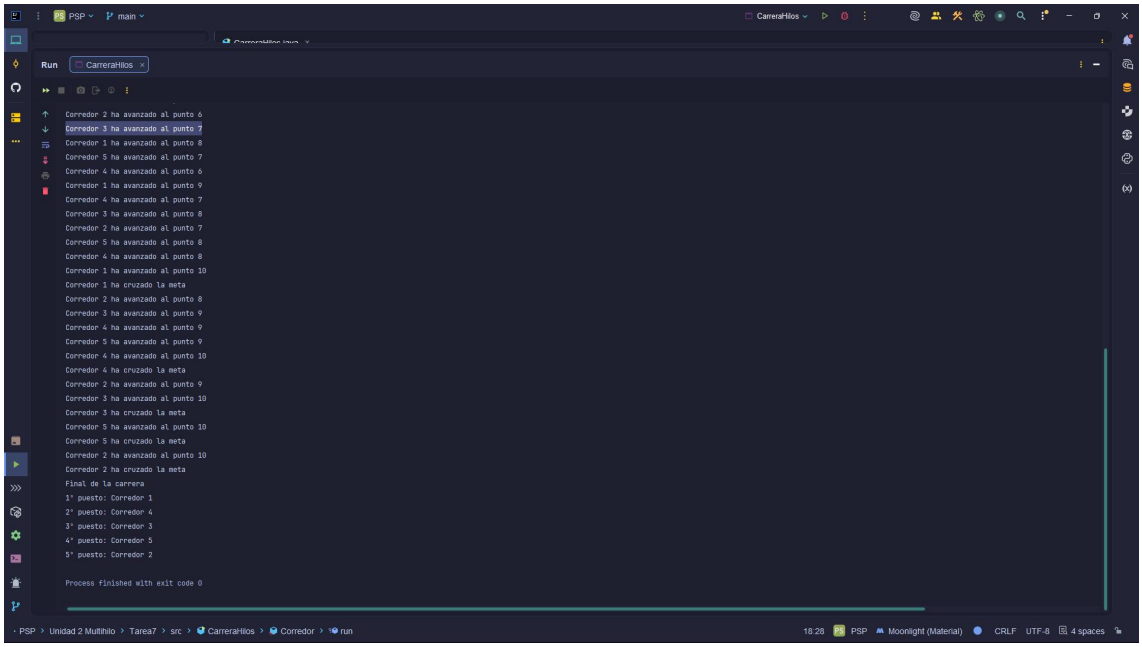
```
        // Mostrar orden
        System.out.println("Final de la carrera");
        synchronized (ordenLlegada) {
            for (int i = 0; i < ordenLlegada.size(); i++) {
                System.out.println((i + 1) + "º puesto: " +
ordenLlegada.get(i));
            }
        }
    }
}
```

ii. Incluye aquí la captura la captura de pantalla de la consola que muestre el orden de llegada de los corredores:

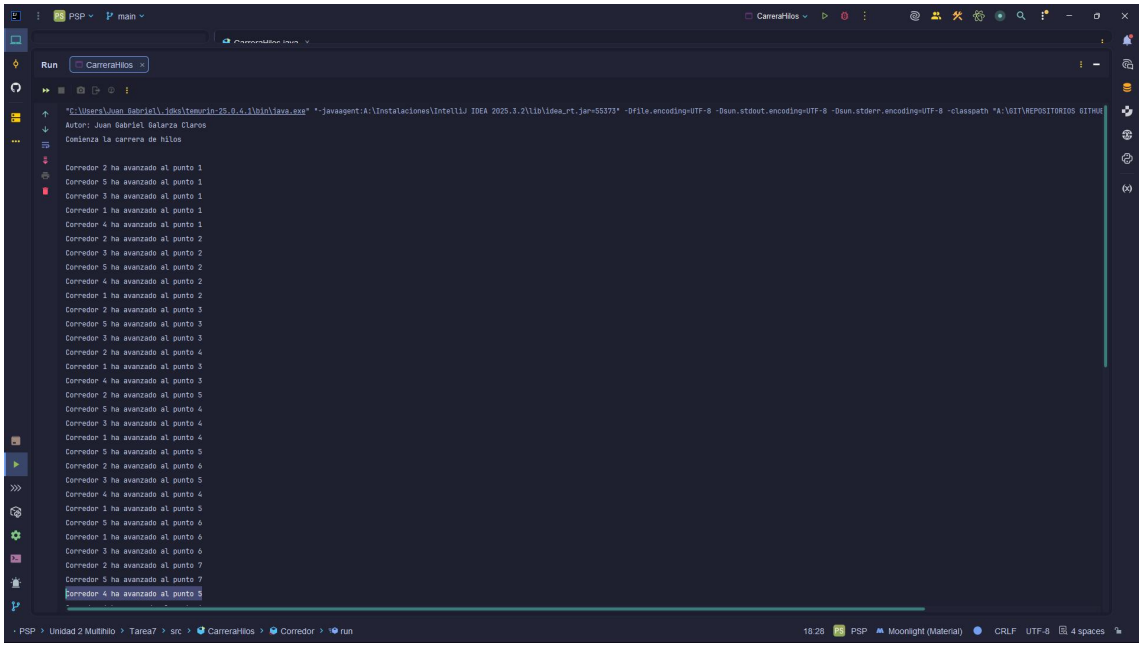
CAPTURA/S DE PANTALLA

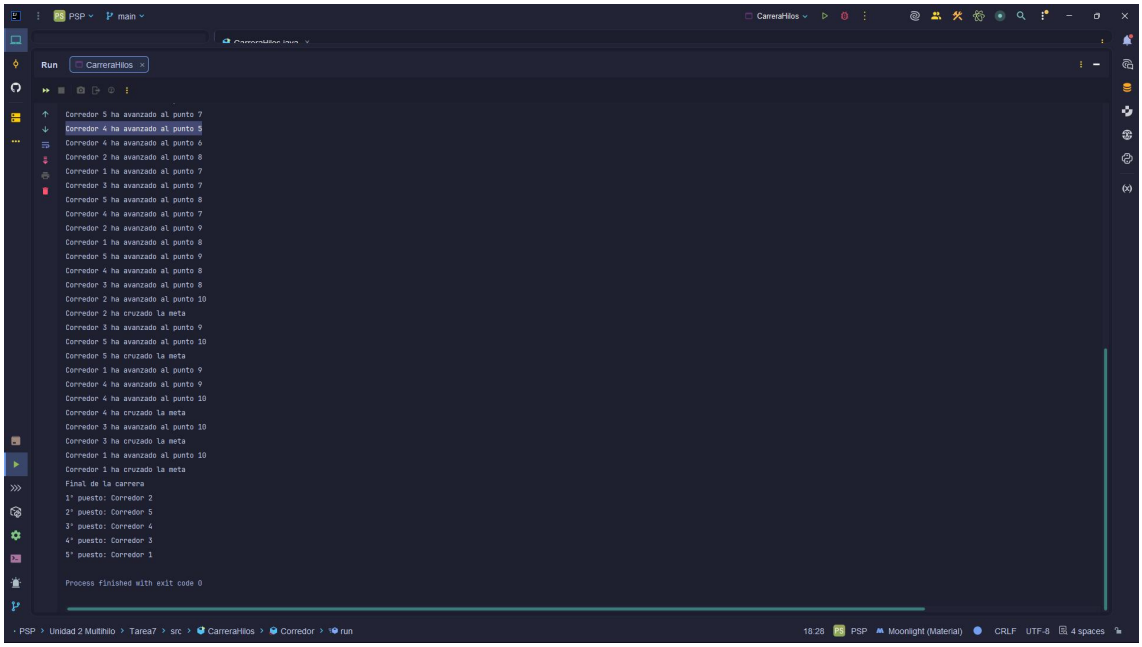
EJECUCIÓN 1:




Corredor 2 ha avanzado al punto 6
Corredor 3 ha avanzado al punto 7
Corredor 1 ha avanzado al punto 8
Corredor 5 ha avanzado al punto 7
Corredor 4 ha avanzado al punto 6
Corredor 1 ha avanzado al punto 9
Corredor 4 ha avanzado al punto 7
Corredor 3 ha avanzado al punto 8
Corredor 2 ha avanzado al punto 7
Corredor 5 ha avanzado al punto 8
Corredor 4 ha avanzado al punto 8
Corredor 1 ha avanzado al punto 10
Corredor 1 ha cruzado la meta
Corredor 2 ha avanzado al punto 8
Corredor 3 ha avanzado al punto 9
Corredor 4 ha avanzado al punto 9
Corredor 5 ha avanzado al punto 9
Corredor 4 ha avanzado al punto 10
Corredor 4 ha cruzado la meta
Corredor 2 ha avanzado al punto 9
Corredor 3 ha avanzado al punto 10
Corredor 3 ha cruzado la meta
Corredor 5 ha avanzado al punto 10
Corredor 5 ha cruzado la meta
Corredor 2 ha avanzado al punto 10
Corredor 2 ha cruzado la meta
Final de la carrera
1° puesto: Corredor 1
2° puesto: Corredor 4
3° puesto: Corredor 3
4° puesto: Corredor 5
5° puesto: Corredor 2
Process finished with exit code 0

EJECUCIÓN 2:


Autor: Juan Gabriel Galarza Claros
Comienza la carrera de hilos
Corredor 2 ha avanzado al punto 1
Corredor 5 ha avanzado al punto 1
Corredor 3 ha avanzado al punto 1
Corredor 1 ha avanzado al punto 1
Corredor 4 ha avanzado al punto 1
Corredor 2 ha avanzado al punto 2
Corredor 3 ha avanzado al punto 2
Corredor 5 ha avanzado al punto 2
Corredor 4 ha avanzado al punto 2
Corredor 1 ha avanzado al punto 2
Corredor 2 ha avanzado al punto 3
Corredor 5 ha avanzado al punto 3
Corredor 3 ha avanzado al punto 3
Corredor 2 ha avanzado al punto 4
Corredor 1 ha avanzado al punto 3
Corredor 4 ha avanzado al punto 3
Corredor 2 ha avanzado al punto 5
Corredor 5 ha avanzado al punto 4
Corredor 3 ha avanzado al punto 4
Corredor 1 ha avanzado al punto 4
Corredor 5 ha avanzado al punto 5
Corredor 2 ha avanzado al punto 6
Corredor 3 ha avanzado al punto 5
Corredor 4 ha avanzado al punto 4
Corredor 1 ha avanzado al punto 5
Corredor 5 ha avanzado al punto 6
Corredor 1 ha avanzado al punto 6
Corredor 3 ha avanzado al punto 6
Corredor 2 ha avanzado al punto 7
Corredor 5 ha avanzado al punto 7
Corredor 4 ha avanzado al punto 5



iii. Responde las siguientes preguntas:

- ¿Los hilos con mayor prioridad siempre ganan?

No, porque la prioridad en Java es solo una sugerencia para el sistema operativo

- ¿Qué sucede si todos tienen la misma prioridad?

El sistema operativo tratará a todos por igual, repartiendo el tiempo de CPU mediante pequeños turnos, y el orden sería impredecible